

Lista de Exercícios — Unidade 3

AVL, Tabelas de Dispersão e Filas de Prioridade

Pedro Nascimento — Algoritmos e Estruturas de Dados I (EDI)

28 de junho de 2026

Observação: as questões de *implementação* foram entregues em arquivos Go:

- `avl/avl.go` — toda a AVL (questões 5, 8, 10, 11, 13, 15, 16, 18, 20);
- `hash/hashtable.go` — Tabela de Dispersão / TAD Map (questões 3, 4, 6);
- `heap/heap.go` — Heap Binária / PriorityQueue (questões 3, 4, 5).

Este documento contém as questões *escritas* e as ilustrações. Em toda a unidade adota-se a convenção do material do professor:

$$bf = \text{altura}(\text{subárvore direita}) - \text{altura}(\text{subárvore esquerda}),$$

de modo que $bf < 0$ indica árvore pesada à **esquerda** e $bf > 0$ pesada à **direita**. A altura de uma folha é 0 e a de uma subárvore vazia é -1 .

AVL

1) O que é fator de balanceamento e como é calculado?

É um valor associado a cada nó que mede o quão desequilibrada está a árvore naquele ponto. Calcula-se como a diferença entre as alturas das subárvores filhas:

$$bf(n) = \text{altura}(n.\text{direita}) - \text{altura}(n.\text{esquerda}).$$

Numa árvore AVL, $bf(n) \in \{-1, 0, +1\}$ para todo nó n . Se $|bf| \geq 2$, o nó está desbalanceado e precisa ser rebalanceado.

2) O que diferencia uma árvore AVL de uma BST?

Uma AVL é uma BST (mantém a propriedade de ordenação: menores à esquerda, maiores à direita), porém com uma restrição adicional de balanceamento: para todo nó, $|bf| \leq 1$. Após cada inserção ou remoção a AVL se reequilibra através de rotações, garantindo que a altura permaneça $O(\log n)$. Uma BST comum não tem essa garantia e pode degenerar.

3) Desempenho de BSTs e AVLs

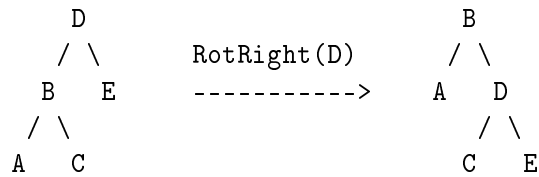
As operações de busca, inserção e remoção em ambas custam $O(h)$, onde h é a altura.

- **BST:** no caso médio (inserções aleatórias) $h \approx \log n$, logo $O(\log n)$. No pior caso (dados já ordenados) a árvore vira uma "lista", $h = n - 1$, e as operações ficam $O(n)$.
- **AVL:** o balanceamento garante $h = O(\log n)$ *sempre*, então todas as operações são $O(\log n)$ no pior caso. O custo extra é manter altura/ bf e eventualmente realizar rotações, ambos $O(1)$ por nó no caminho.

Resumindo: a AVL troca um pequeno custo constante por nó pela garantia de desempenho logarítmico no pior caso.

4) Resultado da rotação do nó D para a direita

Antes (D é a raiz, B seu filho esquerdo com filhos A e C, E seu filho direito):



O filho esquerdo (B) sobe e vira raiz; D desce e passa a ser o filho direito de B; a antiga subárvore direita de B (o nó C) passa a ser a subárvore esquerda de D.

5) Código que rotaciona à direita

Implementado em `avl/avl.go` (método `RotRight`). Versão básica pedida na questão:

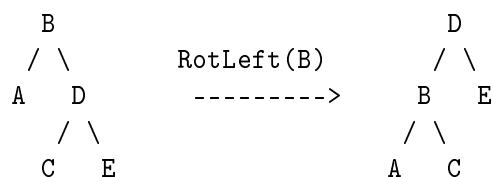
```
func (root *BstNode) RotRight() *BstNode {
    newRoot := root.left
    root.left = newRoot.right
    newRoot.right = root
    return newRoot
}
```

6) Por que é fundamental retornar um `*BstNode`?

Porque a rotação *muda qual nó é a raiz* daquela subárvore. Como os nós são ligados por ponteiros e o pai aponta para o filho via atribuição (`pai.left = filho.RotRight()`), a função precisa devolver o novo nó-raiz para que o pai atualize seu ponteiro. Sem o retorno, o pai continuaria apontando para o antigo topo da subárvore, corrompendo a estrutura.

7) Resultado da rotação do nó B para a esquerda

Antes (B é a raiz, A seu filho esquerdo, D seu filho direito com filhos C e E):



O filho direito (D) sobe e vira raiz; B desce e vira filho esquerdo de D; a antiga subárvore esquerda de D (o nó C) passa a ser a subárvore direita de B.

8) Código que rotaciona à esquerda

Implementado em `avl/avl.go` (método `RotLeft`). Versão básica:

```
func (root *BstNode) RotLeft() *BstNode {
    newRoot := root.right
    root.right = newRoot.left
    newRoot.left = root
    return newRoot
}
```

9) Quais nós atualizar e em qual ordem (questões 4 e 7)

Após uma rotação, apenas os *dois* nós envolvidos têm suas alturas/*bf* alterados, e a ordem importa: deve-se atualizar primeiro o nó que **desceu** (a antiga raiz) e só depois o nó que **subiu** (a nova raiz), pois a altura/*bf* de quem sobe é calculada a partir dos filhos — que incluem o nó que acabou de descer.

- **Questão 4** (RotRight em D): atualizar primeiro **D**, depois **B**.
- **Questão 7** (RotLeft em B): atualizar primeiro **B**, depois **D**.

10) Função que atualiza as propriedades de um nó

Implementado em `avl/avl.go` (método `UpdateProperties`):

```
func (root *BstNode) UpdateProperties() *BstNode {
    lh := height(root.left) // -1 se nil
    rh := height(root.right)
    if lh > rh {
        root.height = lh + 1
    } else {
        root.height = rh + 1
    }
    root.bf = rh - lh
    return root
}
```

11) RotRight e RotLeft com UpdateProperties

Implementado em `avl/avl.go`. As rotações já chamam `UpdateProperties` na ordem descrita na questão 9 (antiga raiz e depois nova raiz):

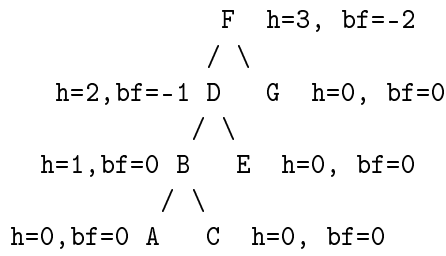
```
func (root *BstNode) RotRight() *BstNode {
    newRoot := root.left
    root.left = newRoot.right
    newRoot.right = root
    root.UpdateProperties() // antiga raiz (desceu)
    newRoot.UpdateProperties() // nova raiz
    return newRoot
}

func (root *BstNode) RotLeft() *BstNode {
    newRoot := root.right
    root.right = newRoot.left
    newRoot.left = root
    root.UpdateProperties()
    newRoot.UpdateProperties()
    return newRoot
}
```

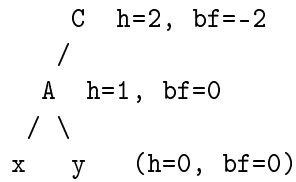
12) Os 6 casos de desbalanceamento

Em cada ilustração o nó desbalanceado tem $|bf| = 2$. Os casos *neutro* surgem tipicamente após remoções.

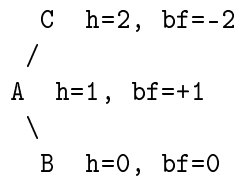
esquerda-esquerda (raiz $bf = -2$, filho esquerdo $bf = -1$) — exemplo do enunciado:



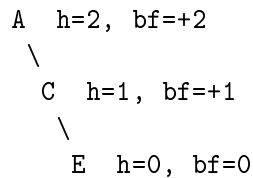
esquerda-neutro (raiz $bf = -2$, filho esquerdo $bf = 0$):



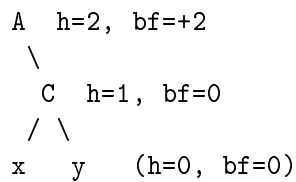
esquerda-direita (raiz $bf = -2$, filho esquerdo $bf = +1$):



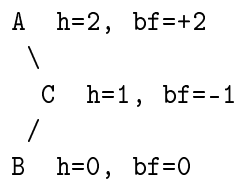
direita-direita (raiz $bf = +2$, filho direito $bf = +1$):



direita-neutro (raiz $bf = +2$, filho direito $bf = 0$):



direita-esquerda (raiz $bf = +2$, filho direito $bf = -1$):



13) Ações para balancear cada caso (descrição)

As funções correspondentes estão em `avl/avl.go`.

- **esquerda-esquerda:** rotação simples à direita no nó desbalanceado (`RotRight`).
- **esquerda-neutro:** rotação simples à direita (`RotRight`).
- **esquerda-direita:** rotação dupla — primeiro `RotLeft` no filho esquerdo, depois `RotRight` no nó desbalanceado.
- **direita-direita:** rotação simples à esquerda (`RotLeft`).
- **direita-neutro:** rotação simples à esquerda (`RotLeft`).
- **direita-esquerda:** rotação dupla — primeiro `RotRight` no filho direito, depois `RotLeft` no nó desbalanceado.

```
func (root *BstNode) RebalanceLeftLeft() *BstNode { return root.
    RotRight() }
func (root *BstNode) RebalanceLeftNeutral() *BstNode { return root.
    RotRight() }
func (root *BstNode) RebalanceLeftRight() *BstNode {
    root.left = root.left.RotLeft()
    return root.RotRight()
}
func (root *BstNode) RebalanceRightRight() *BstNode { return root.
    RotLeft() }
func (root *BstNode) RebalanceRightNeutral() *BstNode { return root.
    RotLeft() }
func (root *BstNode) RebalanceRightLeft() *BstNode {
    root.right = root.right.RotRight()
    return root.RotLeft()
}
```

14) Por que retornar um `*BstNode` nessas funções?

Pelo mesmo motivo da questão 6: o rebalanceamento envolve rotações que trocam a raiz da subárvore. A função devolve o novo topo para que o chamador (o pai, ou a própria `Rebalance`) religue o ponteiro corretamente.

15) Função que rebalanceia uma AVL

Implementado em `avl/avl.go` (método `Rebalance`), reusando as funções da questão 13 e a classificação dos casos da questão 12:

```
func (root *BstNode) Rebalance() *BstNode {
    root.UpdateProperties()
    if root.bf < -1 { // pesada a esquerda
        if root.left.bf < 0 {
            return root.RebalanceLeftLeft()
        } else if root.left.bf > 0 {
            return root.RebalanceLeftRight()
        }
        return root.RebalanceLeftNeutral()
    }
    if root.bf > 1 { // pesada a direita
        if root.right.bf > 0 {
            return root.RebalanceRightRight()
        } else if root.right.bf < 0 {
            return root.RebalanceRightLeft()
        }
    }
}
```

```

        return root.RebalanceRightNeutral()
    }
    return root
}

```

16) Adição em AVL

Implementado em `avl/avl.go` (método `Add`). A diferença para a BST é que, ao retornar pela recursão, cada nó religa o filho ao resultado de `Add` e chama `Rebalance`, devolvendo a nova raiz da subárvore:

```

func (root *BstNode) Add(value int) *BstNode {
    if value <= root.value {
        if root.left == nil {
            root.left = NewNode(value)
        } else {
            root.left = root.left.Add(value)
        }
    } else {
        if root.right == nil {
            root.right = NewNode(value)
        } else {
            root.right = root.right.Add(value)
        }
    }
    return root.Rebalance()
}

```

17) Por que retornar um `*BstNode` no `Add`?

Porque a inserção pode disparar um rebalanceamento (rotação) que altera a raiz da subárvore. Retornando o novo topo, a linha `root.left = root.left.Add(value)` religa o pai ao nó correto após o possível ajuste. É também o que permite tratar de forma uniforme a criação do primeiro nó e o reequilíbrio.

18) Remoção em AVL

Implementado em `avl/avl.go` (método `Remove`). Igual à remoção da BST, mas com `Rebalance` no retorno de cada nível:

```

func (root *BstNode) Remove(value int) *BstNode {
    if root == nil {
        return nil
    }
    if value < root.value {
        root.left = root.left.Remove(value)
    } else if value > root.value {
        root.right = root.right.Remove(value)
    } else {
        if root.left == nil && root.right == nil {
            return nil
        } else if root.left != nil && root.right == nil {
            return root.left
        } else if root.left == nil && root.right != nil {
            return root.right
        } else {

```

```

        maxEsq := root.left.Max()
        root.value = maxEsq
        root.left = root.left.Remove(maxEsq)
    }
}
return root.Rebalance()
}

```

19) Por que retornar um *BstNode no Remove?

Mesma razão: a remoção pode mudar a raiz da subárvore — tanto pelos casos de 0/1 filho (em que se retorna `nil` ou o único filho) quanto pelo rebalanceamento por rotações. O retorno permite ao pai religar seu ponteiro ao nó correto.

20) Função que identifica se uma árvore é AVL

Implementado em `avl/avl.go` (método `IsAVL`):

```

func (root *BstNode) IsAVL() bool {
    if root == nil {
        return true
    }
    bf := realHeight(root.right) - realHeight(root.left)
    if bf < -1 || bf > 1 {
        return false
    }
    return root.left.IsAVL() && root.right.IsAVL()
}

```

21) Inserindo os caracteres do nome (sem repetição)

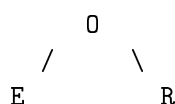
Nome: **Pedro Nascimento**. Caracteres únicos, na ordem de aparição:

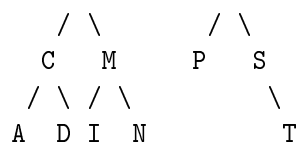
P, E, D, R, O, N, A, S, C, I, M, T

Inserindo um a um numa AVL (comparação alfabética), ocorrem as seguintes operações de rebalanceamento:

- Ao inserir **D** (sequência P,E,D): a raiz P fica $bf = -2$ e E $bf = -1$ — caso *esquerda-esquerda*: `RotRight` em P. Raiz passa a ser E.
- Ao inserir **N**: E fica $bf = +2$ com o filho direito P em $bf = -1$ — caso *direita-esquerda*: `RotRight` em P seguido de `RotLeft` em E. Raiz passa a ser O.
- Ao inserir **S**: o nó P fica $bf = +2$ com filho direito R $bf = +1$ — caso *direita-direita*: `RotLeft` em P.
- Ao inserir **C**: o nó D fica $bf = -2$ com filho esquerdo A $bf = +1$ — caso *esquerda-direita*: `RotLeft` em A seguido de `RotRight` em D.
- Ao inserir **M**: o nó N fica $bf = -2$ com filho esquerdo I $bf = +1$ — caso *esquerda-direita*: `RotLeft` em I seguido de `RotRight` em N.
- Ao inserir **I** e **T**: nenhum rebalanceamento é necessário.

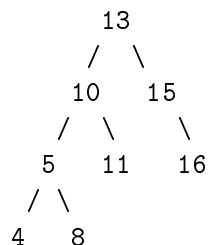
Árvore AVL final (verificada com o código de `avl/avl.go`):



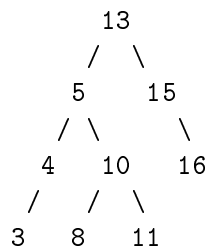


22) Inserção do valor 3 (questão de múltipla escolha)

Árvore inicial:



Inserindo 3, ele desce até virar filho esquerdo de 4. Isso deixa o nó **10** desbalanceado ($bf = -2$, com o filho esquerdo 5 em $bf = -1$): caso *esquerda-esquerda*, resolvido por **RotRight** em 10. O nó 5 sobe ao lugar de 10 (o filho direito de 5, que era 8, passa a ser filho esquerdo de 10). A árvore resultante é:



Resposta: alternativa A. (A alternativa B corresponde à inserção *sem* rebalanceamento, que deixaria a árvore inválida.)

Tabelas de Dispersão

1) Qual é a estrutura de dados básica?

Um **array (vetor)**. Cada posição do array é um *bucket/slot*, e o índice de cada elemento é obtido aplicando-se a função de hash sobre a chave. O acesso direto por índice no array é o que dá o tempo médio $O(1)$ às operações.

2) Como elementos de diferentes tipos são mapeados para a tabela?

Através de uma **função de hash**, que converte a chave — de qualquer tipo — em um número inteiro (*hash code*). Sobre esse inteiro aplica-se o operador módulo pelo tamanho do array para obter um índice válido: `indice = hash(chave) % tamanho`. A função deve ser *determinística* (mesma chave $\rightarrow$ mesmo índice) e distribuir as chaves o mais uniformemente possível. Tipos diferentes (inteiros, strings, objetos) usam estratégias próprias de hashing: inteiros podem usar o próprio valor; strings combinam os códigos dos caracteres (ex.: função polinomial base 31, usada em `hash/hashtable.go`).

3) Funções fundamentais do TAD Map

Implementado em `hash/hashtable.go`:

```
type Map interface {
    Put(key string, value int) // insere/atualiza um par chave-valor
    Get(key string) (int, bool) // valor e se a chave existe
    Remove(key string) bool // remove; true se existia
    Contains(key string) bool // true se a chave esta presente
    Size() int // quantidade de pares
    IsEmpty() bool
}
```

4) Estrutura de dado HashTable

Implementado em `hash/hashtable.go` (encadeamento separado):

```
type entry struct { // no da lista ligada de um bucket
    key string
    value int
    next *entry
}
type HashTable struct {
    buckets []*entry // array de buckets
    size int // numero de pares armazenados
}
```

5) O que são colisões e como endereçá-las?

Uma **colisão** ocorre quando duas chaves *distintas* produzem o *mesmo* índice na tabela (algo inevitável, pois o espaço de chaves costuma ser maior que o de índices). Principais formas de tratamento:

- **Encadeamento separado (separate chaining)**: cada bucket guarda uma lista ligada (ou outra estrutura) com todas as entradas que caíram naquele índice. É a abordagem usada na implementação.
- **Endereçamento aberto (open addressing)**: todas as entradas ficam no próprio array; em caso de colisão procura-se outro slot livre por uma sequência de sondagem:

- *sondagem linear* — tenta $i + 1, i + 2, \dots$;
- *sondagem quadrática* — tenta $i + 1^2, i + 2^2, \dots$;
- *hash duplo* — usa uma segunda função de hash para o passo.

Em qualquer caso, manter o *fator de carga* (elementos/tamanho) baixo, redimensionando a tabela quando necessário, é o que preserva o desempenho médio $O(1)$.

6) Código das funções da questão 3

Implementado em `hash/hashtable.go` (métodos `Put`, `Get`, `Remove`, `Contains`, `Size`, `IsEmpty`, além da função interna `hash`).

Filas de Prioridade (Heaps Binários)

1) O que é uma Fila de Prioridade e o que é uma Heap? Aplicações

Uma **Fila de Prioridade** é um TAD parecido com uma fila, mas em que cada elemento tem uma *prioridade*; a remoção (*poll*) sempre retorna o elemento de maior prioridade, e não o que entrou primeiro (não é FIFO). Uma **Heap (binária)** é a estrutura de dados mais usada para implementá-la: é uma *árvore binária completa* (preenchida da esquerda para a direita) que satisfaz a *propriedade de heap* — todo pai tem prioridade maior ou igual à dos filhos. Costuma ser armazenada em um array, onde, para o índice i , os filhos estão em $2i + 1$ e $2i + 2$ e o pai em $\lfloor (i - 1)/2 \rfloor$.

Aplicações: algoritmo de Dijkstra (caminho mínimo) e Prim (árvore geradora mínima), codificação de Huffman, *heapsort*, escalonamento de processos por prioridade, simulação de eventos discretos e seleção dos k maiores/menores elementos.

2) Diferença entre Heap de mínimo e Heap de máximo

- **Heap de mínimo (min-heap):** todo pai é $\leq$ seus filhos; a raiz é o *menor* elemento. Poll retorna o menor.
- **Heap de máximo (max-heap):** todo pai é $\geq$ seus filhos; a raiz é o *maior* elemento. Poll retorna o maior.

A única diferença prática é o sentido da comparação usada para subir/descer elementos — na implementação, o método `higherPriority` encapsula isso através do campo `isMax`.

3) Funções fundamentais do tipo PriorityQueue

Implementado em `heap/heap.go`:

```
type PQ interface {
    Add(value int)           // insere um elemento
    Peek() (int, bool)      // consulta o de maior prioridade
    Poll() (int, bool)      // remove e retorna o de maior prioridade
    Remove(value int) bool  // remove uma ocorrência de um valor
    Size() int
    IsEmpty() bool
}
```

4) Estrutura de dado BinaryHeap

Implementado em `heap/heap.go`:

```
type BinaryHeap struct {
    data []int // heap armazenada em array
    isMax bool // false = min-heap, true = max-heap
}
```

5) Código das funções da questão anterior

Implementado em `heap/heap.go` (métodos `Add`, `Peek`, `Poll`, `Remove`, `Size`, `IsEmpty`, e os auxiliares `siftUp/siftDown`).

6) Heap de Mínimo com os 10 primeiros caracteres do nome

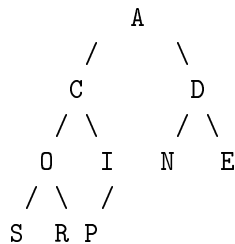
Os 10 primeiros caracteres de **Pedro Nascimento** são:

P, E, D, R, O, N, A, S, C, I

Inserindo-os nessa ordem numa min-heap (menor letra = maior prioridade), obtém-se o array (verificado com `heap/heap.go`):

[A, C, D, O, I, N, E, S, R, P]

que corresponde à árvore:



Operações (cada uma aplicada à heap acima):

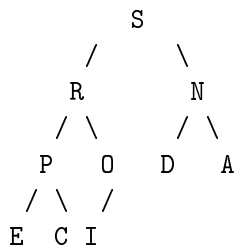
- **Adicionar A:** A entra no fim e sobe até quase a raiz (empata com o A da raiz, que permanece). Resultado: [A, A, D, O, C, N, E, S, R, P, I].
- **Adicionar Z:** Z é o maior, então fica como folha no fim, sem subir. Resultado: [A, C, D, O, I, N, E, S, R, P, Z].
- **Poll:** remove a raiz A (menor). O último elemento sobe para a raiz e desce reorganizando. Resultado: [C, I, D, O, P, N, E, S, R]. (removido: A).
- **Remover P** (primeira letra do nome): P estava na última posição; é removido diretamente. Resultado: [A, C, D, O, I, N, E, S, R].

7) Heap de Máximo com os 10 primeiros caracteres do nome

Mesmos caracteres P, E, D, R, O, N, A, S, C, I, agora numa max-heap (maior letra = maior prioridade). Array resultante (verificado com `heap/heap.go`):

[S, R, N, P, O, D, A, E, C, I]

correspondente à árvore:



Operações (cada uma aplicada à heap acima):

- **Adicionar A:** A é pequeno, fica como folha no fim sem subir. Resultado: [S, R, N, P, O, D, A, E, C, I, A].
- **Adicionar Z:** Z é o maior de todos, sobe desde a folha até virar a raiz. Resultado: [Z, S, N, P, R, D, A, E, C, I, O].
- **Poll:** remove a raiz S (maior). O último elemento sobe e desce reorganizando. Resultado: [R, P, N, I, O, D, A, E, C]. (removido: S).
- **Remover P** (primeira letra do nome): troca-se P pelo último elemento (I) e ajusta-se. Resultado: [S, R, N, I, O, D, A, E, C].